

159.102 Notes

Section 5 : Programming the MASSEY machine

Reminder – The Instruction Set for the MASSEY Machine

Each instruction consists of 16 bits written as 4 hexadecimal digits. The first digit is the **opcode**. The other 3 digits are the **operands**. The list of machine instructions follows.

- 1xYZ** **Load** register Rx with the value YZ. (Note 2-digit hex limit on number size)
e.g. 1213 means load register R2 with the hexadecimal number 13 (19 in decimal).
- 20xy** **Load** register Rx with the number in register Ry.
e.g. 20A7 means load register RA (register 10 in decimal) with the number in register R7.
- 3xYZ** **Load** register Rx with the number in the memory location at address YZ.
e.g. 34AB means load register R4 with the contents from memory location AB.
- 4xYZ** **Store** the number in register Rx in the memory location at address YZ.
e.g. 45B1 means store the contents of register R5 in the memory location B1.
- 6xyz** **Add** the numbers in registers Ry and Rz and load the result into register Rx.
Note: this is **integer addition** using 2's complement arithmetic.
e.g. 6823 means add the contents of R2 and R3 and load the result into R8.
- 70x0** **Negate** register Rx. (Carry out complement and add 1 – in effect, multiply Rx with –1)
e.g. 70A0 means multiply the number in RA by –1.
- 8xYZ** **Shift** the number in register Rx **right** by YZ bits
e.g. 8403 means shift the number in R4 3 bits to the right.
Registers store 16 bits, so there is no point shifting them by 16 or more bits.
Thus in practice, Y will be 0.
- 9xYZ** **Shift** the number in register Rx **left** for YZ bits
e.g. 9602 means shift the number in R6 2 bits to the left.
- Axyz** Bitwise **AND** the numbers in registers Ry and Rz and load the result into register Rx.
e.g. A045 means bitwise AND the numbers in R4 and R5 and load the result into R0.
- Bxyz** Bitwise **OR** the numbers in registers Ry and Rz and load the result into register Rx.
e.g. BC74 means bitwise OR the numbers in R7 and R4 and load the result into R12.
- Cxyz** Bitwise **XOR** the numbers in registers Ry and Rz and load the result into register Rx.
e.g. C5F3 means bitwise XOR the numbers in RF and R3 and load the result into R5.
- DxYZ** **Jump** to the instruction at address YZ **if** the value in register Rx is equal to the value in register R0.
e.g. D43C means the following:
 (a) compare the contents of R4 with R0.
 (b) if they are equal, load 3C into the **program counter**.
- E000** **Halt**

Assembler code

A program written in machine code is hard to read – it is just a list of hexadecimal numbers. To make machine code easier to read and write, it can be written in **assembler code** (also called “assembly language”).

For example, the machine code 1234 (“load register R2 with the value 34 hexadecimal”) is written in assembler code as LOAD R2, 34.

Just as each computer model has its own machine code with its own instruction set, each computer model also has its own assembler code. One machine instruction corresponds exactly to one line of assembler code. We can also write the instructions in “C++-like” code – this is not assembler code, but it can be an aid to understanding and writing assembler code.

Assembler code for the MASSEY machine

Machine code	Assembler code	Explanation	C++ equivalent
1xYZ	LOAD Rx, YZ	Load register Rx with the value YZ.	Rx = 0xYZ; // 0x indicates a hex constant
20xy	LOAD Rx, Ry	Load register Rx with the number in register Ry.	Rx = Ry;
3xYZ	LOAD Rx, [YZ]	Load register Rx with the number in the memory location at address YZ.	Rx = memory[YZ];
4xYZ	STORE Rx, [YZ]	Store the number in register Rx in the memory location at address YZ.	memory[YZ] = Rx;
6xyz	Rx = Ry + Rz	Add the numbers in registers Ry and Rz and load the result into register Rx.	Rx = Ry + Rz;
70x0	NEGATE Rx	Negate register Rx.	Rx = -Rx;
8xYZ	SHIFT Rx RIGHT YZ	Shift the number in register Rx right by YZ bits	Rx = Rx >> 0xYZ;
9xYZ	SHIFT Rx LEFT YZ	Shift the number in register Rx left for YZ bits	Rx = Rx << 0xYZ;
Axyz	Rx = Ry AND Rz	Bitwise AND the numbers in registers Ry and Rz and load the result into register Rx.	Rx = Ry & Rz;
Bxyz	Rx = Ry OR Rz	Bitwise OR the numbers in registers Ry and Rz and load the result into register Rx.	Rx = Ry Rz;
Cxyz	Rx = Ry XOR Rz	Bitwise XOR the numbers in registers Ry and Rz and load the result into register Rx.	Rx = Ry ^ Rz;
DxYZ	JUMP [YZ] IF Rx=R0	Jump to the instruction at address YZ if the value in register Rx is equal to the value in register R0.	// rough equivalent while (Rx == R0){}
E000	HALT	Halt	exit();

Example 1: a program to do addition

Address	Contents
00	3508
01	3609
02	6056
03	400A
04	E000
05	0000
06	0000
07	0000
08	001B
09	0008
0A	0000

The machine code is hard to read!

Here is the same code with parallel version in assembler code and C++.

Address	Contents	Assembler	“C++”
00	3508	LOAD R5, [08]	R5 = memory[08];
01	3609	LOAD R6, [09]	R6 = memory[09];
02	6056	R0 = R5 + R6;	R0 = R5 + R6;
03	400A	STORE R0, [0A]	memory[0A] = R0;
04	E000	HALT	
05	0000		
06	0000		
07	0000		
08	001B		
09	0008		
0A	0000		

Working – show the changes in the PC, Instruction Register, Registers and Memory:

Program Counter: 00, 01, 02, 03, 04

Instruction Register: 3508, 3609, 6056, 400A, E000

Registers: R5 = 1B, R6 = 08, R0 = 23

Memory: store 23 in 0A

Extra working: $1B + 8 = 23$ (decimal equivalent is $27 + 8 = 35$)

An actual C++ program for this could be:

```
int main(){
    int num1, num2, result;
    num1 = 27;
    num2 = 8;
    result = num1 + num2;
}
```

Note that the C++ compiler takes care of where each variable is stored in memory. In machine code or assembler code, you have to keep track of the memory locations yourself.

Example 2: A program to do subtraction

Address	Contents
00	11FC
01	1211
02	7020
03	6012
04	4007
05	E000
06	0000
07	0000

Still hard to read! Here is the same code with assembler and C++.

Address	Contents	Assembler	“C++”
00	11FC	LOAD R1, FC	R1 = 0xFC;
01	1211	LOAD R2, 11	R2 = 0x11;
02	7020	NEGATE R2	R2 = -R2;
03	6012	R0 = R1 + R2	R0 = R1 + R2;
04	4007	STORE R0, [07]	memory[07]=R0;
05	E000	HALT	
06	0000		
07	0000		

Working: show the changes in the PC, Instruction Register, Registers and Memory:

Program Counter: 00, 01, 02, 03, 04, 05

Instruction Register: 11FC, 1211, 7020, 6012, 4007, E000

Registers: R1 = FC, R2 = 11 then -11 (stored in 2s complement as EF), R0 = EB

Memory: store EB in 07

Extra working: in hex: FC + (-11) = EB
in decimal: 252 - 17 = 235

Equivalent C++ code: int result = 252 - 17;

Example 3: A program to modify instructions while the program is running

Address	Contents	Assembler	“C++”
00	1404	LOAD R4, 4	R4 = 4;
01	1505	LOAD R5, 5	R5 = 5;
02	1066	LOAD R0, 66	R0 = 0x66;
03	9008	SHIFT R0 LEFT 8	R0 = R0 << 8;
04	1145	LOAD R1, 45	R1 = 0x45;
05	6201	R2 = R0 + R1	R2 = R0 + R1;
06	4207	STORE R2, [07]	memory[07] = R2;
07	0000	????	????
08	460A	STORE R6, [0A]	memory[0A] = R6;
09	E000	HALT	
0A	0000		
0B	0000		

Working: show the changes in the PC, Instruction Register, Registers and Memory:

(**Bold** indicates the instruction that has been created by the program while it runs)

Program Counter: 00, 01, 02, 03, 04, 05, 06, 07, 08, 09

Instruction Register: 1404, 1505, 1066, 9008, 1145, 6201, 4207, **6645**, 460A, E000

Registers: R4 = 04, R5 = 05, R0 = 66... 6600, R1 = 45, R2 = 6645, R6 = 09

Memory: **store 6645 in 07**, store 09 in 0A

Extra working: shift 66 left by 2 hex digits = 6600 (all in hex)
6600 + 45 = 6645 (all in hex)

What is the equivalent C++ code?

Memory location 07 started off as 0000 but an instruction (6645) was placed there while the program was running. This is not possible in C++ or other high-level programming languages.

Example 4: Programming an IF-ELSE statement

Address	Contents	Assembler	“C++”	Notes
00	1113	LOAD R1, 13	R1 = 0x13;	= 19 decimal
01	1212	LOAD R2, 12	R2 = 0x12;	= 18 decimal
02	7020	NEGATE R2	R2 = -R2;	= -18 decimal
03	6312	R3 = R1 + R2	R3 = R1 + R2;	now R3 = 1
04	1480	LOAD R4, 80	R4 = 0x80;	= 128 decimal
05	9408	SHIFT R4 LEFT 8	R4 <<= 8;	8 bits = 2 hex digits; now R4 = 0x8000
06	A534	R5 = R3 AND R4	R5 = R3 & R4;	set all except the first bit of R3 to 0; detects if R3 is non-negative
07	1000	LOAD R0, 00	R0 = 0;	
08	D50B	JUMP [0B] IF R5==R0		jump if R3>=0
09	1101	LOAD R1, 01	R1 = 1;	
0A	D00C	JUMP [0C] IF R0==R0		Always jump
0B	1102	LOAD R1, 02	R1 = 2;	
0C	410E	STORE R1, [0E]	memory[0E] = R1;	
0D	E000	HALT		
0E	0000			

Working – show the changes in the PC, Instruction Register, Registers and Memory:

(**Bold** indicates a JUMP instruction and the destination of the JUMP)

Program Counter: 00, 01, 02, 03, 04, 05, 06, 07, 08, **0B**, 0C, 0D

Instruction Register: 1113, 1212, 7020, 6312, 1480, 9408, A534, 1000, **D50B**, 1102, 410E, E000

Registers: R1 = 13... 02, R2 = 12...-12, R3 = 01, R4 = 80...8000, R5 = 00, R0 = 00

Memory: store 02 in 0E

Extra working: shift 80 left by 2 hex digits = 8000 (all in hex)
0001 AND 8000 = 0000
if (R5 == R0) then set PC to 0B

What is the equivalent C++ code?

```
int num, result;

num = 19;
if (num >= 18) {
    result = 2;
} else {
    result = 1;
}
```

Note 1: (num >= 18) is true if num – 18 is a non-negative number. How do we know if a number is positive? It will have a zero in the sign bit.

Note 2: the machine code for the “true section” of the if statement (load R1 with 2) comes AFTER the machine code for the “false section” (load R1 with 1). This may make it confusing to read.

Example 5: Programming a LOOP

Consider the following fragment of C++ code that contains a loop. In this example we’ll implement this loop in assembler and machine code.

```
int main() {
    int x, y;
    x = 0;
    while (x < 10) {
        x = x + 3;
    }
    y = x;
}
```

In machine code, all the work takes place in the registers. For this reason, machine code programs often start off by loading useful values into certain registers. When the program is running, these registers remain unchanged and can be used when required at different points in the program. In this particular program, the following registers are loaded at the start:

R0 is set to zero to be used by the JUMP instruction

RF is set to 3 because in this example we are calculating $x = x + 3$

RE is set to ten (0A) because in this example we are checking `if (x < 10)`

RD uses shift to set a mask with a 1 on the left and the rest is zero (to check the sign bit)

In the machine code, variable x will be stored in memory location 65 and variable y will be stored in memory location 75.

Address	Contents	Assembler	“C++”	Notes
00	1000	LOAD R0, 0	R0 = 0;	to be used for jumps
01	1F03	LOAD RF, 3	RF = 3;	to be used to add 3
02	1E0A	LOAD RE, 0A	RE = 0x0A;	10 decimal, for loop
03	1D80	LOAD RD, 80	RD = 0x80;	
04	9D08	SHIFT RD LEFT 8	RD <<= 8;	RD now 0x8000, to be used to check sign bit
05	1100	LOAD R1, 0	R1 = 0;	
06	4165	STORE R1, [65]	memory[0x65]=R1;	variable x stored in memory location 65 hex
07	3165	LOAD R1, [65]	R1=memory[0x65];	start of loop
08	2021	LOAD R2, R1	R2 = R1;	
09	7020	NEGATE R2;	R2 = -R2;	
0A	622E	R2 = R2 + RE;	R2 = R2 + RE;	i.e. R2 = 10-x
0B	A32D	R3 = R2 AND RD;	R3 = R2 & RD;	if R2>=0 then R3=0
0C	D30E	JUMP [0E] IF R3==R0		jump if x<10
0D	D011	JUMP [11] IF R0==R0		always jump
0E	611F	R1 = R1 + RF	R1 = R1 + RF;	x = x + 3;
0F	4165	STORE R1, [65]	memory[0x65] = R1;	update value of x
10	D007	JUMP [07] IF R0==R0		jump back to start of test for loop
11	4175	STORE R1, [75]	memory[0x75]=R1;	y = x;
12	E000	HALT		

Optimisation of machine code

Optimisation is a technique designed to increase the speed and performance of machine code. The basic approach is to look for redundant machine code instructions and remove them from the program. If you took great care when designing and creating your machine code, then optimisation would not be needed. However, we often create machine code one section at a time and it is easy to overlook where instructions become redundant. Let us look at optimisation in the loop program above. I will use the assembler code and not the machine code, just because the assembler code is easier to read.

In this program, the first instruction for the test of the loop is this:
LOAD R1 with the contents of memory location 65

The last line of the body of the loop (before it jumps back to repeat the test) is this:
STORE R1 in memory location 65 (update the value of variable x)

When the program is running, each time the loop ends, it jumps back to the test. This means that these two lines are actually executed in this order:

STORE R1 in memory location 65 (update the value of variable x)
LOAD R1 with the contents of memory location 65

The LOAD instruction is redundant as R1 already contains the value that is in memory location 65. We can thus remove the LOAD instruction from the program – one less instruction to be executed.

The main use of optimisation is inside compilers. Compilers (a) translate your high-level instructions into machine code, and then (b) optimise the machine code. This means that your final executable program (in machine code) will be as efficient as possible.

Comparing machine code with high level languages

All numbers are in binary (or hexadecimal). Machine code does not use decimal numbers.

There are no if-statements and loops. Everything is done with JUMP.

Simple tests (e.g. $x < 10$) are complicated to calculate in machine code.

There is a C++ equivalent of JUMP called goto. Do not use goto in C++ programs. Jumping is used at the machine code level and makes high level programs difficult to read (and easy to make a mistake).

Machine code often uses instructions such as SHIFT and AND.

Variables are never used directly in machine code. E.g. $y = x$; can only be done by loading x into a register and then storing the register into y.

Machine code is generally more complicated to use, harder to read and you are more likely to make mistakes (and then it is more difficult to fix the errors).

Machine code can change instructions while the program is running. This is because data and instructions are actually the same thing (they are all binary numbers). High level programs usually cannot do this because they separate the instructions from the data. Some people argue that to have “true” artificial intelligence we need programs that can change their instructions while they are running. Is this a good idea...?

If written by an expert, machine code will usually be more efficient (and thus faster to execute) than high level code.

Think how far we have come in this course already. Everything has been built up from tiny components in layers.

Layer 1: a single logic gate (built from valves or transistors) can do a simple operation, eg compute AND on two input bits

Layer 2: several logic gates together can form a memory bit and do more complex operations, eg addition of multi-bit numbers

Layer 3: machine code enables computers to carry out long sequences of operations, including loops, allowing very large calculations

Layer 4: assembler code is easier to write and can be easily translated directly into machine code. (I wonder... would you write that translation program in machine code or in assembler code?)

Layer 5: High-level languages (like C++) are compiled into machine code, often via intermediate assembler-like languages. The resulting code is highly optimised. In theory, this level makes it practical to compute almost anything. But regardless, we move onto...

Layer 6: Vast code libraries do many standard tasks for you; higher-level security features (some of which are in C++) make coordinating large software projects more reliable.

Layer 7: AI writes its own code. But how do you know the code is correct?

Example 6: Integer multiplication

Notice that the MASSEY machine does not contain any machine code instruction for integer multiplication. That means to multiply numbers we will have to develop an algorithm for multiplication that only uses instructions available in the MASSEY machine code.

The first idea that comes to mind is to use repeated addition. For example, $3 \times 5 = 5 + 5 + 5 = 15$, which can be programmed using a loop. But that would be extremely slow for large numbers like 2345×6789 .

Instead, we recall the algorithm for long multiplication that is taught in primary school. Here's an example:

$$\begin{array}{r} 234 \\ \times 221 \\ \hline 234 \\ + 4680 \\ + 46800 \\ \hline 51714 \end{array}$$

Now try out the same algorithm in binary on the test case $3 \times 5 = 15$:

$$\begin{array}{r} 11 \\ \times 101 \\ \hline 11 \\ + 000 \\ + 1100 \\ \hline 1111 \end{array}$$

The algorithm is much simpler in binary!

- There is no “multiplication table” to memorize – we only have to multiply digits by 0 or 1
- Multiplying by 1 means just copying the number, and shifting bits to the left to take account of the place value
- The binary algorithm will loop through the bits of the second factor, inspecting each bit in turn. If it is a 1 we will add the (shifted) first factor to the running total of the product.
- The shifts will be simplified if we shift the first factor one bit left (ready for adding to the running total if needed) and shift the second factor one bit to the right one bit each time through the loop (so that we'll be testing the rightmost bit (bit 1)) each time.

Here's the algorithm implemented in C++, but using only low-level and bitwise operations suitable for translating into assembler code.

```
#include <iostream>
using namespace std;

int main(){
    unsigned int R0, R1, R2, R3, R4;

    cout << "Enter two integers: ";
    cin >> R1 >> R2;
    R3 = 0;    // R3 will hold the running total of the product R1*R2
    while (R2 != 0){    // Loop until all the non-zero bits of R2 have
                        // been shifted to the right and disappeared
        if ((R2 & 1) != 0){ // If the last bit of R2 is 1
            R3 += R1;    // Add R1 to the running total of the product
        };
        R1 <<= 1;    // Shift R1 1 bit to the left
        R2 >>= 1;    // Shift R2 1 bit to the right
    }
    cout << "Product is " << R3 << endl;
}
```

Finally, we translate into assembler and then machine code. The final algorithm is quite short – the main loop is only 7 lines of machine code!

Machine code for integer multiplication in the MASSEY machine

Memory	Contents	Assembler	“C++”	notes
00	1000	LOAD R0, 0	R0 = 0;	
01	1501	LOAD R5, 1	R5 = 1;	Used to extract last bit
02	1103	LOAD R1, 3	R1 = 3;	First factor
03	1205	LOAD R2, 5	R2 = 5;	Second factor: 3×5
04	1300	LOAD R3, 0	R3 = 0;	Running total for product
05	A425	R4 = R2 AND R5	R4 = R2 & R5;	Last bit of R2
06	D408	JUMP TO 08 IF R4==R0		If last bit is 0, skip ahead
07	6331	R3 = R3 + R1	R3 += R1;	
08	9101	SHIFT R1 LEFT 1	R1 <<= 1;	
09	8201	SHIFT R2 RIGHT 1	R2 >>= 1;	
0A	D20C	JUMP TO 0C IF R2==R0		finished?
0B	D005	JUMP TO 05 IF R0==R0		always jump back
0C	E000	HALT		

Definitions

High level language – a programming language designed to overcome the problems of programming in machine code. Usually by making instructions more readable and easier to use. Examples of high-level languages include C++, C, C#, Java and Python.

Machine code – instructions in binary using the instruction set for a particular computer. Usually stored in .exe files (i.e. the code can be immediately executed).

Optimisation of machine code – instructions removing redundant machine code instructions to make a program more efficient because (a) they take up less memory (there are fewer instructions) and (b) the program runs faster (there are less instructions to execute).

Assembler code – a low level programming language in which instructions usually correspond directly to the machine code instructions. Usually stored in .asm files.

Assembler – a program that translates assembler code into machine code.

Compiler – a program that translates high level code into machine code and then optimises the machine code.